SESSION 8

JOINS: INNER, LEFT & RIGHT

FOUNDATION SESSION | SQL PRACTICE ANSWERS

Goal: Solve all five tasks using simple MySQL-compatible SQL, grey code boxes, sample output, and easy explanations.

Database note: The examples use a separate database named session8_join_practice to avoid conflicts with existing tables. Task 1 intentionally does not add a foreign key from Orders to Users, so Task 4 can demonstrate an order whose user_id has no matching user.

Join concepts in this session

Join type	What it keeps
INNER JOIN	Only rows that match in both tables.
LEFT JOIN	Every row from the left table, plus matching rows from the right table. No match on the right gives NULL.
RIGHT JOIN	Every row from the right table, plus matching rows from the left table. No match on the left gives NULL.

Task 1 — Create Users and Orders Tables

Create the Users and Orders tables, insert at least three users and five orders, and make sure at least one user has no orders. The sample data also includes an order with user_id = 99, which does not exist in Users; this will help test Task 4.

SQL code

```
CREATE DATABASE IF NOT EXISTS session8_join_practice;
USE session8_join_practice;

CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    username VARCHAR(100) NOT NULL,
    city VARCHAR(50)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    product VARCHAR(100),
    amount DECIMAL(10,2)
);

INSERT INTO Users (user_id, username, city)
VALUES
    (1, 'Ahad', 'Ahmedabad'),
    (2, 'Priya', 'Mumbai'),
    (3, 'Rahul', 'Surat');

INSERT INTO Orders (order_id, user_id, product, amount)
VALUES
    (101, 1, 'Pizza', 250.00),
    (102, 1, 'Burger', 150.00),
    (103, 2, 'Pasta', 300.00),
    (104, 2, 'Sandwich', 120.00),
    (105, 99, 'Biryani', 400.00);

SELECT * FROM Users;
SELECT * FROM Orders;
```

What the code does

- `CREATE DATABASE` creates a separate practice database; `USE` selects it.
- `Users` stores each user's ID, username, and city. `user_id` is the primary key, so it uniquely identifies each user.
- `Orders` stores each order's ID, user ID, product, and amount. `order_id` is its primary key.
- Ahad has two orders, Priya has two orders, and Rahul has no orders.
- The order for user_id 99 has no matching user. No foreign key is added to Orders, so this sample can demonstrate the unmatched-order case in Task 4.

Sample data overview

Users	City
Ahad	Ahmedabad
Priya	Mumbai
Rahul	Surat

order_id	user_id	product	amount
101	1	Pizza	250.00
102	1	Burger	150.00
103	2	Pasta	300.00
104	2	Sandwich	120.00

order_id	user_id	product	amount
105	99	Biryani	400.00

Task 2 — INNER JOIN: Users Who Have Orders

Use INNER JOIN to list usernames and the products they ordered. Only users who have at least one matching order appear in the result.

```
USE session8_join_practice;

SELECT
    u.username,
    o.product
FROM Users AS u
INNER JOIN Orders AS o
    ON u.user_id = o.user_id
ORDER BY o.order_id;
```

Explanation

- `Users AS u` gives the Users table the short alias `u`. `Orders AS o` gives Orders the alias `o`.
- `ON u.user_id = o.user_id` matches each order to the user with the same ID.
- `INNER JOIN` keeps only matching records. Rahul is not shown because he has no order, and the Biryani order is not shown because user_id 99 is missing from Users.

Expected output for the sample data

username	product
Ahad	Pizza
Ahad	Burger
Priya	Pasta
Priya	Sandwich

Task 3 — LEFT JOIN: All Users and Their Orders

Use LEFT JOIN to display every user, including users who have not placed an order. For a user with no order, the product column will show NULL.

```
SELECT
    u.username,
    o.product
FROM Users AS u
LEFT JOIN Orders AS o
    ON u.user_id = o.user_id
ORDER BY u.user_id, o.order_id;
```

Explanation

- `Users` is on the left side of the JOIN, so every row from Users is retained.
- When a user has matching orders, their order products appear. A user can appear on multiple rows if they have multiple orders.
- Rahul appears with NULL because there is no order where `user_id = 3`. The unmatched order for `user_id = 99` is not included because the LEFT JOIN starts from Users.

Expected output for the sample data

username	product

username	product
Ahad	Pizza
Ahad	Burger
Priya	Pasta
Priya	Sandwich
Rahul	NULL

Task 4 — RIGHT JOIN: All Orders and Usernames

Use RIGHT JOIN to keep all orders and display the matching username wherever a user exists. If an order refers to a user_id that is not present in Users, the username will be NULL.

```
SELECT
    o.order_id,
    o.user_id,
    u.username,
    o.product,
    o.amount
FROM Users AS u
RIGHT JOIN Orders AS o
    ON u.user_id = o.user_id
ORDER BY o.order_id;
```

Explanation

- `Orders` is on the right side of the JOIN, so every order is retained.
- When an order has a matching user ID, the username is displayed.
- Order 105 has `user_id = 99`, but no such user exists. The order still appears, and `username` is NULL.

Expected output for the sample data

order_id	user_id	username	product	amount
101	1	Ahad	Pizza	250.00
102	1	Ahad	Burger	150.00
103	2	Priya	Pasta	300.00
104	2	Priya	Sandwich	120.00
105	99	NULL	Biryani	400.00

Task 5 — Customer Segments and Total Order Amount

Create CustomerSegments, connect it to Users with a foreign key, and show every username with their segment name and total order amount. LEFT JOIN keeps users who have no orders; COALESCE displays 0 instead of NULL for their total.

Step A: Create segments and add the foreign key

```
CREATE TABLE CustomerSegments (
    segment_id INT PRIMARY KEY,
    segment_name VARCHAR(50) NOT NULL
);

INSERT INTO CustomerSegments (segment_id, segment_name)
VALUES
    (1, 'Regular'),
    (2, 'Premium'),
    (3, 'New Customer');

ALTER TABLE Users
```

```
ADD COLUMN segment_id INT;

UPDATE Users
SET segment_id = CASE user_id
    WHEN 1 THEN 2
    WHEN 2 THEN 1
    WHEN 3 THEN 3
END;

ALTER TABLE Users
ADD CONSTRAINT fk_users_segment
FOREIGN KEY (segment_id)
REFERENCES CustomerSegments(segment_id);
```

Step B: Show customer segment and total order amount

```
SELECT
    u.username,
    cs.segment_name,
    COALESCE(SUM(o.amount), 0) AS total_order_amount
FROM Users AS u
INNER JOIN CustomerSegments AS cs
    ON u.segment_id = cs.segment_id
LEFT JOIN Orders AS o
    ON u.user_id = o.user_id
GROUP BY u.user_id, u.username, cs.segment_name
ORDER BY u.user_id;
```

Explanation

- `CustomerSegments` stores the available segment IDs and names, such as Regular, Premium, and New Customer.
- `Users.segment_id` stores each user's segment. The foreign key ensures a non-NULL segment_id must exist in CustomerSegments.
- `INNER JOIN CustomerSegments` retrieves the matching segment name for each user.
- `LEFT JOIN Orders` keeps users even when they have no orders.
- `SUM(o.amount)` adds the amounts of all matched orders for a user. `COALESCE(..., 0)` returns 0 for a user who has no orders.
- `GROUP BY` returns one result row per user and segment.

Expected output for the sample data

username	segment_name	total_order_amount
Ahad	Premium	400.00
Priya	Regular	420.00
Rahul	New Customer	0.00

The orphan order for user_id 99 is not included in these user totals because it does not match a row in Users.

Final Revision: Which JOIN Should You Use?

JOIN	Use it when you want...
INNER JOIN	Only records that have matching rows in both tables.
LEFT JOIN	All records from the left table, even if no match exists on the right.
RIGHT JOIN	All records from the right table, even if no match exists on the left.

Quick memory tip: INNER = matches only; LEFT = keep everything on the left; RIGHT = keep everything on the right.